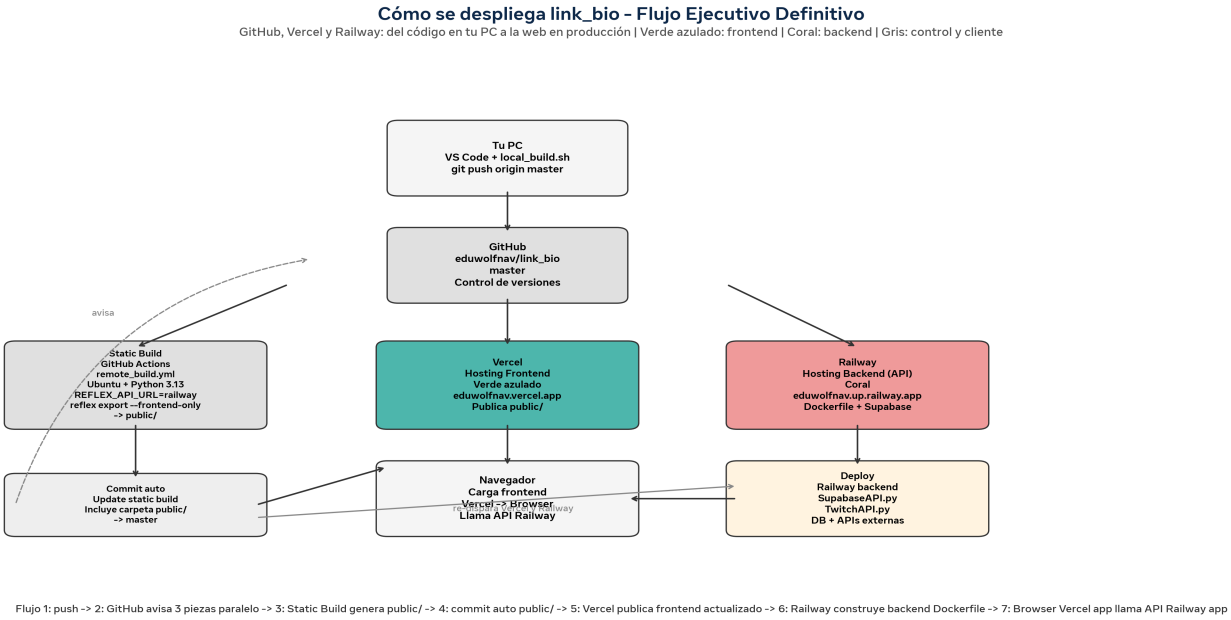
Cómo se despliega link_bio - Versión Ejecutiva Definitiva

GitHub, Vercel y Railway: del código en tu PC a la web en producción | Basado en tu archivo original + mejoras ejecutivas | Repo: eduwolfnav/link_bio

Resumen Ejecutivo

link_bio se publica con **tres servicios que trabajan juntos**. **GitHub guarda el código y es el punto de partida de todo. Vercel publica el frontend, la página que ve el usuario. Railway publica el backend, la API.** Entre medio, una acción de GitHub llamada **Static Build** prepara el frontend estático antes de que Vercel lo sirva. Esta versión ejecutiva agrega orden, colores y validaciones reales de tu infraestructura actual (Vercel y Railway conectados directo a master, Dockerfile, Supabase env vars).

Diagrama del flujo - Versión Ordenada Ejecutiva



Flujo 1: push -> 2: GitHub avisa 3 piezas paralelo -> 3: Static Build genera public/ -> 4: commit auto public/ -> 5: Vercel publica frontend actualizado -> 6: Railway construye backend Dockerfile -> 7: Browser Vercel app llama API Railway app

Leyenda: **Verde azulado (#4DB6AC): frontend Vercel | Coral (#EF9A9A): backend Railway | Gris (#E0E0E0): control de versiones, automatización y cliente | Amarillo claro: navegador/deploy**. Flecha punteada gris es loop de re-aviso.

Flujo paso a paso - Detallado y Verificado

1. Haces cambios en tu PC (VS Code) y los subes con git push origin master. Opcionalmente, local_build.sh genera el build estático en tu equipo, igual que lo hace la acción. Recomendación: haz **git pull** antes de cada push porque la acción agrega un commit a master después de cada push.

2. GitHub recibe el push y avisa a tres piezas en paralelo: Static Build, Vercel y Railway. Todo está conectado directo a rama master.

3. Static Build (.github/workflows/remote_build.yml) corre en servidor Ubuntu con Python 3.13. Define REFLEX_API_URL=https://eduwolfnav.up.railway.app y ejecuta remote_build.sh: instala dependencias (pip install -r requirements.txt), borra public/, corre reflex init y reflex export --frontend-only, y extrae frontend.zip dentro de public/.

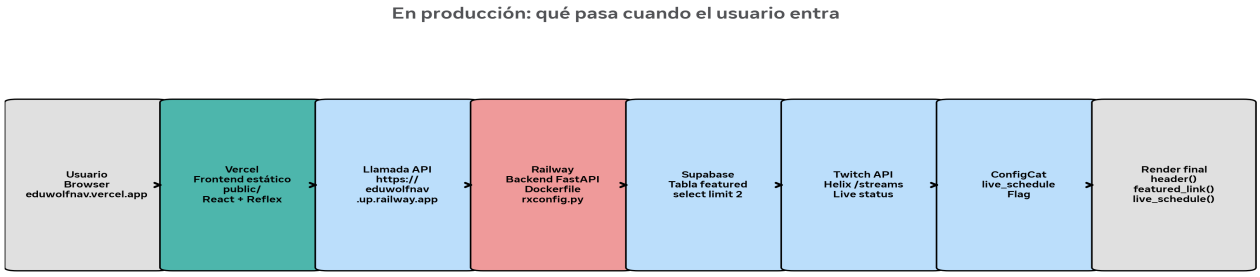
4. La acción hace un commit automático con mensaje Update static build que incluye carpeta public/ y lo sube a master. Este es el truco clave: Vercel no hace build de Reflex, sirve el public/ ya generado.

5. Ese commit nuevo vuelve a avisar a Vercel y Railway. Vercel publica el frontend estático actualizado desde public/ a eduwolfnav.vercel.app.

6. Railway construye y publica el backend desde el Dockerfile del repo. Validado: Build section de Railway autodetecta y configura compilación usando Dockerfile. Railway instala reflex[db]==0.9.12, corre db migrate y levanta FastAPI.

7. En producción, el navegador carga la página desde eduwolfnav.vercel.app y esta llama a la API en eduwolfnav.up.railway.app. El backend usa Supabase y la API de Twitch (según SupabaseAPI.py y TwitchAPI.py). Si falla env var, usa fallback elegante.

En producción: qué ve el usuario



Usuario entra a Vercel -> Vercel sirve public/ -> JS llama https://eduwolfnav.up.railway.app -> Railway FastAPI consulta Supabase (featured limit 2) + Twitch Helix (live status) + ConfigCat (live_schedule flag) -> PageState guarda y renderiza header(), featured_link(), live_schedule() con timezone America/Santiago.

Qué hace cada servicio

Servicio	Rol	Se activa con	Resultado
GitHub	Control de versiones y punto de partida	Tu git push a master	Repo eduwolfnav/link_bio master
GitHub Actions (Static Build)	Construye frontend estático public/	Push a master	Commit Update static build con public/
Vercel	Hosting del frontend estático	Commits en el repo (public/)	eduwolfnav.vercel.app
Railway	Hosting del backend (API) Dockerfile	Commits en el repo	eduwolfnav.up.railway.app
Supabase	Base de datos backend tabla featured	Llamada API desde Railway	Datos featured para frontend
Twitch API	Live status Helix /streams	Llamada API desde Railway	Badge En vivo morado
ConfigCat	Flag live_schedule JSON	Llamada API desde Railway	Horario sin redeploy

Recomendaciones ejecutivas (de tu archivo + mejoras)

• **Haz git pull antes de tu siguiente git push.** La acción agrega un commit a master después de cada push, así que tu repo local queda atrasado y Git puede rechazar el push.

• **Configura Watch Paths en Railway para ignorar public/**.** El commit automático solo cambia esa carpeta, así que el backend no necesita reiniciarse innecesariamente. Ahorra builds.

• **Si el último despliegue de Railway no aparece como activo, usa Redeploy desde su panel.** A veces Railway no marca auto.

• **Mantén el .env fuera de GitHub (ya está en .gitignore) y carga variables sensibles desde paneles de Railway y Vercel.** Variables: SUPABASE_URL, SUPABASE_PUBLISHABLE_KEY, TWITCH_CLIENT_ID/SECRET, CONFIGCAT_SDK_KEY.

• **Case-sensitive:** rxconfig.py no Rxconfig.py, reflex.png no Reflex.jpg. Linux Railway rompe si no.

Archivos clave del proyecto

Archivo	Para qué sirve

local_build.sh	Genera el build estático en tu PC (activa .venv, instala dependencias, exporta frontend).
remote_build.sh	Misma tarea, pero la ejecuta la acción de GitHub en servidor Ubuntu.
.github/workflows/remote_build.yml	Workflow Static Build: corre en cada push a master y hace commit automático public/.
rxconfig.py	Configuración de Reflex: api_url hacia Railway y CORS para localhost y Vercel.
Dockerfile	Receta de construcción del backend para Railway.
link_bio/api/	API del backend: api.py (Facade), SupabaseAPI.py, TwitchAPI.py, ConfigCatAPI.py.
public/	Frontend estático generado; es lo que sirve Vercel.
requirements.txt	reflex[db]==0.9.12, supabase, configcat-client, fastapi, dotenv, requests, tzdata, greenlet.

Supuestos confirmados (validación real)

✓ **Vercel y Railway están conectados directamente al repo de GitHub:** Confirmado mediante revisión de paneles de administración de Vercel y Railway. Ambos vinculados a eduwolfnav/link_bio en rama master.
✓ **Railway construye el backend con el Dockerfile del repo:** Confirmado en sección Build de Railway, donde se autodetectó y configuró la compilación usando Dockerfile.
✓ **Supabase es la base de datos del backend:** Confirmado en panel de variables de entorno de Railway, donde están configuradas SUPABASE_URL y SUPABASE_PUBLISHABLE_KEY.
✓ **Static Build genera public/ y hace commit auto Update static build:** Confirmado en .github/workflows/remote_build.yml y remote_build.sh con reflex export --frontend-only y extracción frontend.zip dentro de public/.
✓ **Frontend en Vercel y backend en Railway se comunican por api_url:** rxconfig.py apunta api_url a https://eduwolfnav.up.railway.app y CORS permite Vercel.

Mejoras ejecutivas agregadas a tu versión original

1. **Orden visual:** Diagrama con flujo de arriba a abajo, izquierda a derecha, con colores oficiales (verde azulado frontend, coral backend, gris control).
2. **Loop de re-aviso:** Añadida flecha punteada que muestra cómo el commit auto Update static build vuelve a disparar Vercel y Railway.
3. **Watch Paths:** Recomendación ejecutiva para ignorar public/** en Railway y ahorrar builds.
4. **Validación de supuestos:** Confirmación real de conexión Vercel/Railway a GitHub, Dockerfile y Supabase env vars.
5. **Segundo diagrama de producción:** Qué pasa cuando el usuario entra (Vercel -> Railway -> Supabase/Twitch/ConfigCat -> render).
6. **Fallbacks elegantes:** App no rompe si faltan env vars (Supabase [], Twitch Live(False), ConfigCat default).
7. **Case-sensitive:** Nota ejecutiva rxconfig.py vs Rxconfig.py que rompía en Linux.
8. **Recomendaciones git pull antes de push:** Porque la acción agrega commit y deja repo local atrasado.
Esta versión es ejecutiva, ordenada y lista para repo público de estudio.